CS 423
Operating System Design:
Persistence:
FS Design and Crash Consistency
Nov 10

Ram Alagappan

AGENDA / LEARNING OUTCOMES

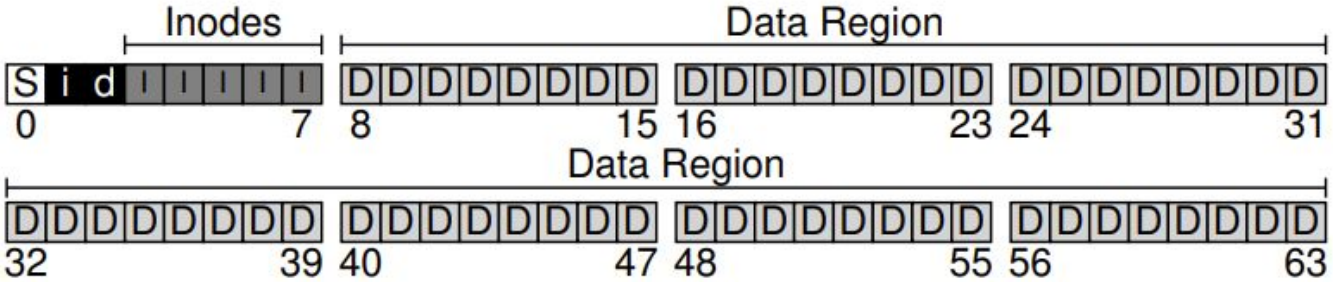
Finish FS design

Crash consistency

- problem
- different solutions

RECAP

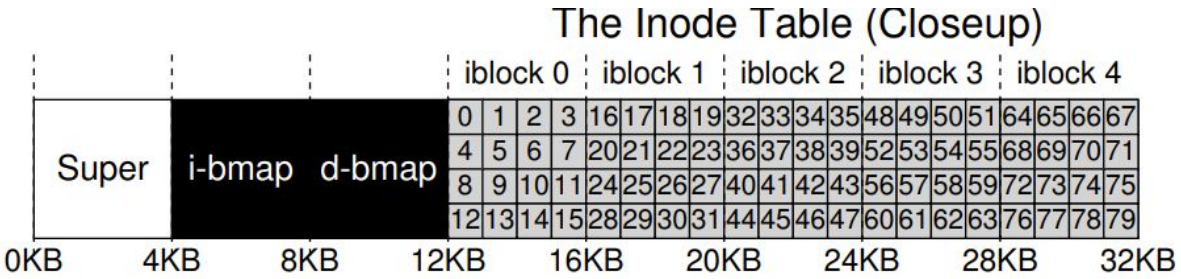
VSFS



Data blocks

Metadata blocks: super, inode bitmap, inodes, data bitmap

INODE



Implicitly know the block + offset from the inode number
inode 16 is at offset 0 of iblock1 (1st block of inode table)

WHAT DOES AN INODE CONTAIN?

Size	Name	What is this inode field for?
2	mode	can this file be read/written/executed?
2	uid	who owns this file?
4	size	how many bytes are in this file?
4	time	what time was this file last accessed?
4	ctime	what time was this file created?
4	mtime	what time was this file last modified?
4	dtime	what time was this inode deleted?
2	gid	which group does this file belong to?
2	links_count	how many hard links are there to this file?
4	blocks	how many blocks have been allocated to this file?
4	flags	how should ext2 use this inode?
4	osd1	an OS-dependent field
60	block	a set of disk pointers (15 total)
4	generation	file version (used by NFS)
4	file_acl	a new permissions model beyond mode bits
4	dir_acl	called access control lists

A few interesting fields:

size

mtime

links_count

block – 15 4-byte pointers

LINK AND UNLINK

An inode can be pointed to by many names!

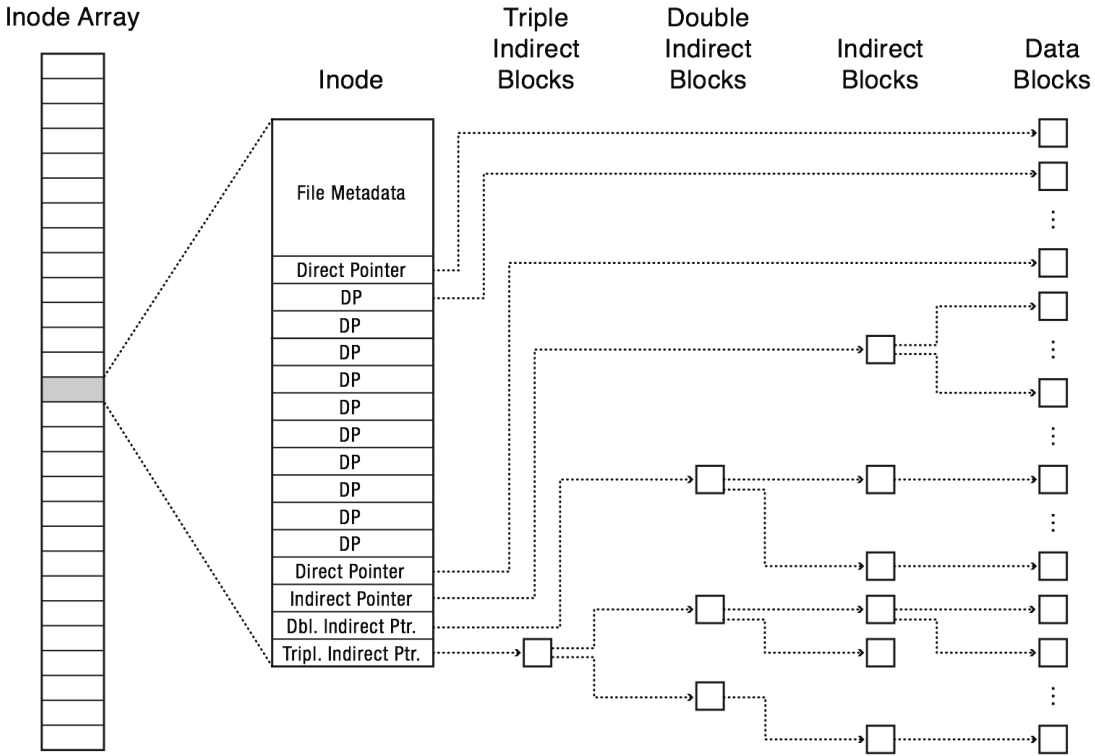
links_count in inode = # of such links to the inode

Link syscall in Linux

Delete a filename: which syscall in Linux?

Delete and free the data blocks of inode when no name points to it

DIRECT AND INDIRECT POINTERS



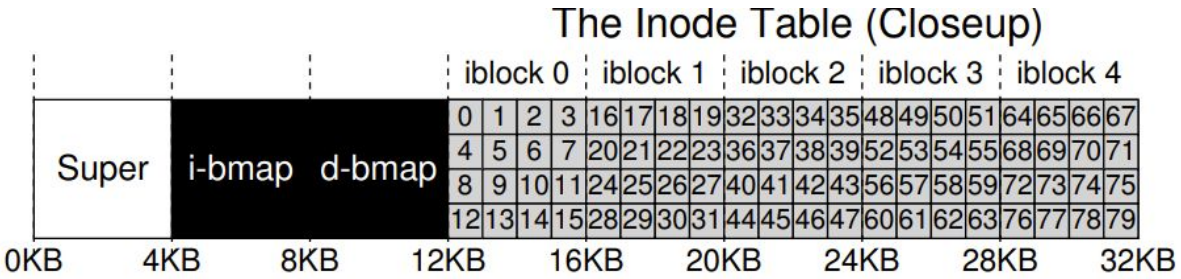
DIRECTORY ORGANIZATION

inum	reclen	strlen	name
5	12	2	.
2	12	3	..
12	12	4	foo
13	12	4	bar
24	36	28	foobar_is_a_pretty_longname

What is the inode of this directory?

What is the inode of its parent?

DIRECTORY ORGANIZATION



Where is a directory's content stored?

No special block type for it !?

END RECAP

CREATING AND WRITING FILE (TIME GOES DOWNWARDS)

	data bitmap	inode bitmap	root inode	foo inode	bar inode	root data	foo data	bar data [0]	bar data [1]
create (/foo/bar)	B	read write	read	read	read write D write	read	read A	write C	
write()	read write				read E write			write	
write()	read write				read write				write

Why read foo data? A

Why read inode bitmap? B

What's written in foo data? C

Why write foo inode? D

Why write bar inode on data write? E

CREATING AND WRITING FILE (TIME GOES DOWNWARDS)

	data bitmap	inode bitmap	root inode	foo inode	bar inode	root data	foo data	bar data [0]	bar data [1]
create (/foo/bar)	B read write		read	read D write	read write	read	read A write C		
write()	read write				read write E			write	
write()	read write				read write				write

How do we know the root's inode number to bootstrap !?

PAGE CACHE

Disk access is expensive

Can cache blocks in memory – all FS do this

Integrated with virtual memory

    can balance fs cache vs. vm

Also helps write buffering

Flushing deamon

FSYNC

Writes are buffered in memory
How to force to disk?
Call fsync…
If app doesn't call, flushed periodically

Q: if app calls write() successfully and then app crashes, will the data become durable?
Q: if app calls fwrite() successfully and then app crashes, will the data become durable?
Q: if app calls write() successfully and then OS crashes, will the data become durable?
Q: if app calls write() + fsync() successfully and then OS crashes, will the data be durable?
Q: if app calls fwrite() + fflush() and then app crashes, will the data become durable?
Q: if app calls fwrite() + fflush() and then OS crashes, will the data become durable?

CRASH CONSISTENCY

Basic problem:

Must update many data structure on disk as a unit

What if failure happens in the middle

Types of failure:

- kernel panic
- power failures

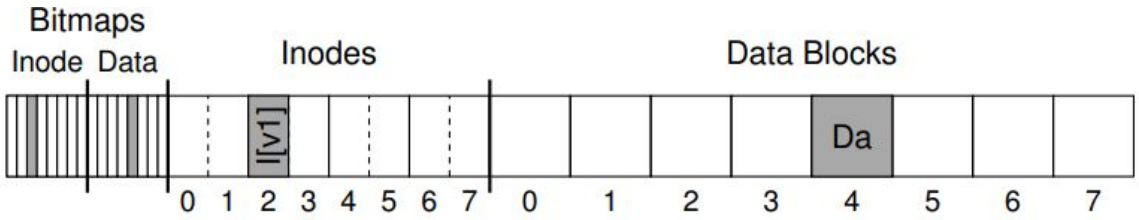
CREATE FILE AND APPEND A BLOCK EXAMPLE

Let's say we create a file and append a block to it…

How many blocks do we need to write to accomplish this?

Which ones?

CREATE AND APPEND A BLOCK EXAMPLE



inode bitmap – allocate the inode for the created file

inode bitmap – allocate the inode for the created file

Data block – the actual data block

inode – modify the inode (change size, add pointer)

PROBLEMS

Now let's say we append a block b, we then need to write the data block itself, the updated inode, the data bitmap

What if only Db is written? Is this a space leak?

Only i[V2] is written to disk? (2 problems)

Data bitmap is alone written to disk?

Bitmap and data are written:

Data and inode are written:

Bitmap and inode are written:

What's special about some cases?

METADATA VS. DATA

FS Metadata consistency vs. Data consistency

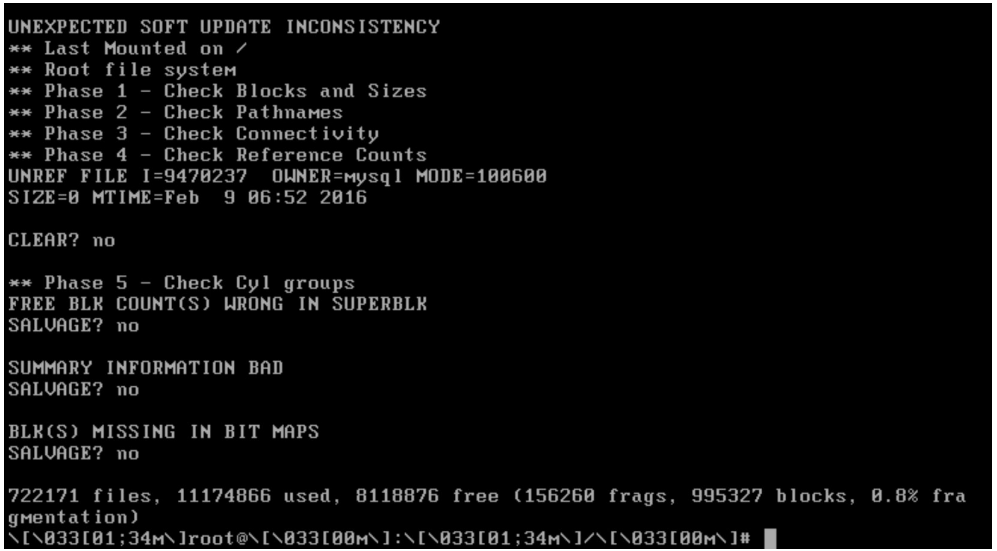
FS metadata consistency: internal structures agree with each other

Data consistency: additionally, the data must “make sense” to applications and users

Let’s look at some solutions for crash consistency

FSCK

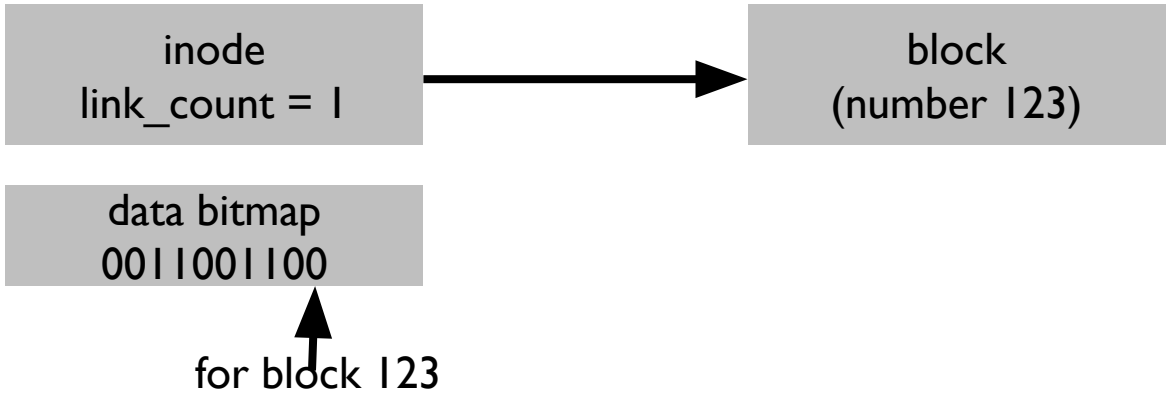
Let inconsistencies happen and take care during reboot



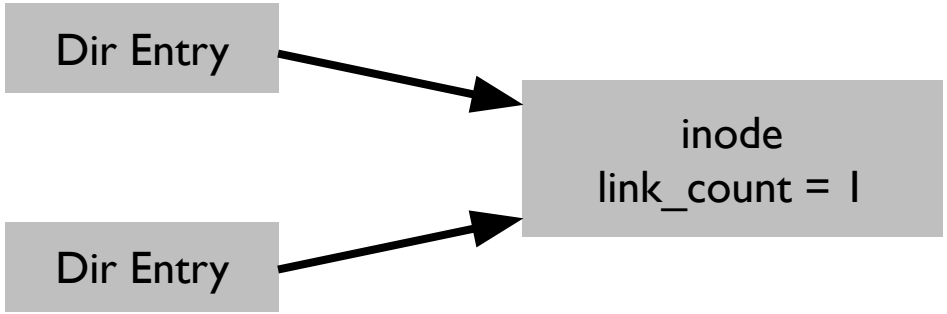
FSCK CHECKS

Do superblocks match?
Is the list of free blocks correct?
Do number of dir entries equal inode link counts?
Do different inodes ever point to same block?
Are there any bad block pointers?
Do directories contain “.” and “..”?
…

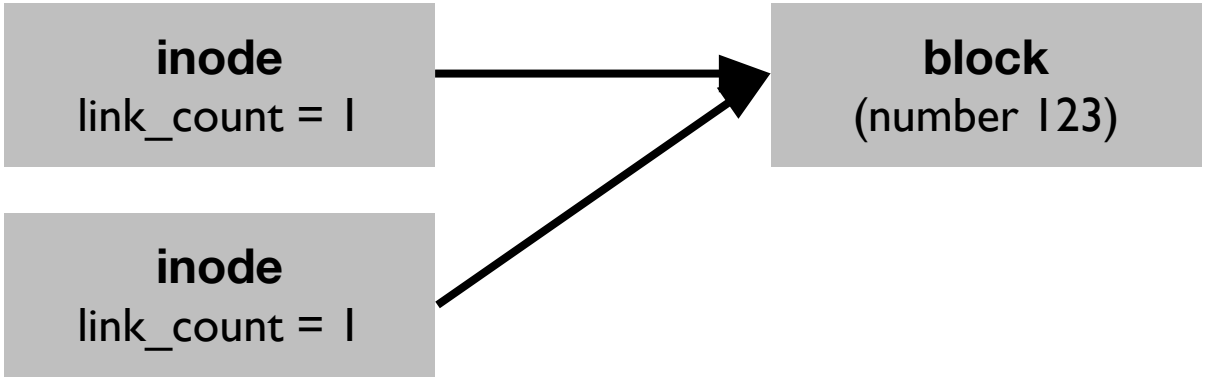
FREE BLOCKS EXAMPLE



LINK COUNT EXAMPLE



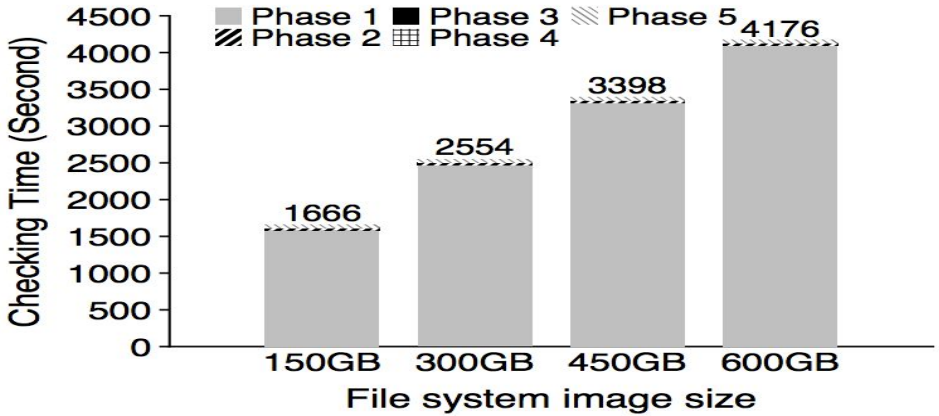
DUPLICATE POINTERS



FSCK PROBLEMS

Not always obvious how to fix file system image - don't know "correct" state, just consistent one

Simply too slow!



Checking a 600GB disk takes ~70 minutes

ffsck: The Fast File System Checker
Ao Ma, Chris Dragga, Andrea C. Arpaci-Dusseau, and Remzi H. Arpaci-Dusseau

JOURNALING OR WAL

Main idea: write a “note” to a well-known location before actually writing the blocks

If crash, know what to fix and how to do so from the note (instead of scanning the entire disk)

JOURNALING IN LINUX EXT3



Append a block to an existing file example

Journal Transaction



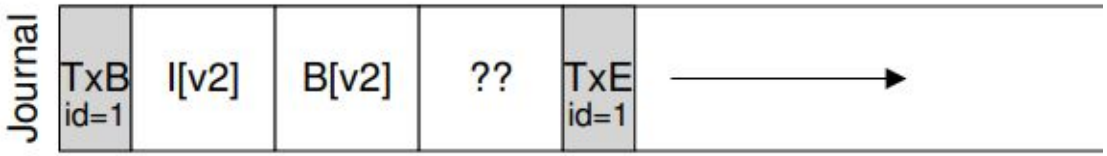
Data journaling vs. metadata journaling

JOURNALING OR WAL

First write the txn to journal
Once that is safe, write the actual blocks (this is called checkpointing)

What if crash happens during journal write?

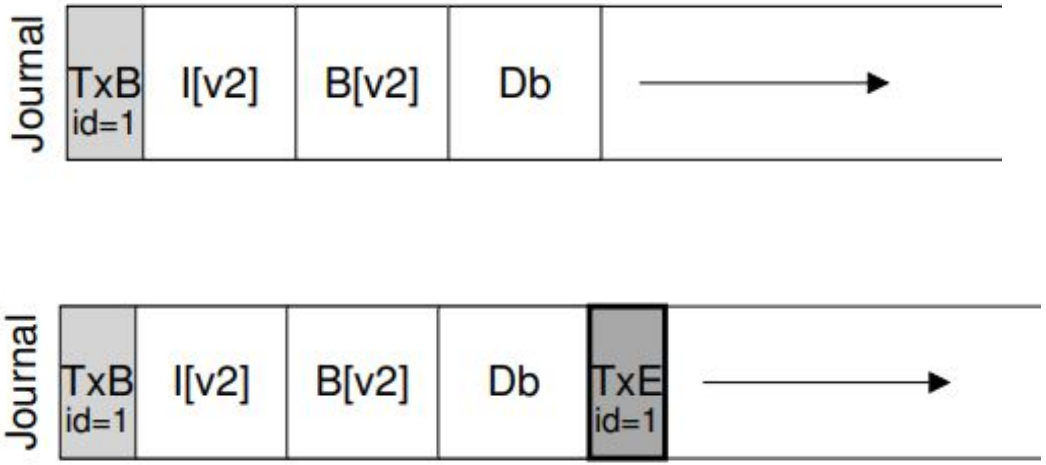
JOURNAL WRITES



How to solve this?

Can issue one write at a time but is too slow
Must maximize how many writes can be concurrently sent
But writing all 5 blocks together is problematic

ONE SOLUTION



Barriers

Incurs a wait or flush between TxB + Rest and TxE…

How to do without waiting?

SOLUTION WITHOUT WAIT

RECOVERY

Scan the journal

Checkpoint completed transactions

Discard otherwise

Will the system be safe if crash happens during recovery?

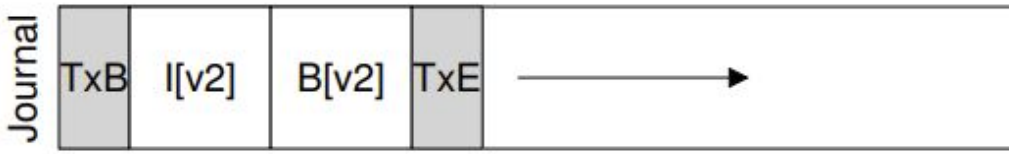
BATCHING FOR EFFICIENCY

WHAT IS THE PROBLEM WITH DJ?

Think about performance…

Which workload will suffer the most?

METADATA JOURNALING



Data blocks directly written to “FS proper” (in place)
Only metadata goes via journal

What is the order of writes?

ORDER OF WRITES

D: data block
JM: metadata blocks in journal
JC: journal commit block
M: metadata block checkpoint
→ means flushes (a→ b means there is a flush between a and b, ensuring that if b is present, then a will be present)
|| means concurrent (a || b means a and b written in parallel and so you can find a, or b, or both a and b

ORDER OF WRITES

D □ JM □ JC □ M

First data, write metadata to journal, write commit block, then checkpoint metadata

Is this safe?

ORDER OF WRITES

D || JM □ JC □ M

Is this safe?

DJ VS. OJ

Data journaling is slow…
Why would someone use it?

What benefits does it provide over metadata journaling

NEXT LECTURE

Finish crash consistency

Log-structured file systems